

and enable user-driven interactions within their AR/VR applications.

Mobile and web app development using Amplify

To develop and deploy the React Native and React web applications using AWS Amplify, you can follow these general steps.

1. Set up an AWS Amplify project.
 - Install the Amplify CLI globally by running `npm install -g @aws-amplify/cli`.
 - In each project's root directory (React Native and React web), initialise Amplify by running `amplify init`.
 - Follow the prompts to configure your Amplify project, including selecting an AWS profile, providing a name for the project, and specifying the environment.
2. Add authentication to your projects.
 - For each project, run `amplify add auth` to add authentication to your projects. Follow the prompts to configure your authentication settings, such as selecting the authentication type, specifying the user pool name, and setting up the required configuration.
3. Develop your React Native app.
4. Develop your React web app.
5. Set up your AWS Amplify backend.
 - For each project, navigate to its respective directory (React Native and React web).
 - Run `amplify push` to deploy the backend resources for each project, including the authentication user pool and any other resources defined in the Amplify configuration.
 - Follow the prompts and confirm the changes to proceed with the deployment.
6. Publish your apps using AWS Amplify.
 - In the AWS Amplify Console, navigate to the 'Apps' section and click 'Connect app'.
 - For each project (React Native and React web), select the

appropriate option (React Native or React) and choose your preferred repository service.

- Connect each app's repository and configure the build settings for React Native and React web separately.
- After configuring the build settings for each app, click 'Next' and follow the instructions to deploy your React Native and React web apps using the AWS Amplify console.

Make sure to refer to the AWS Amplify documentation for more detailed instructions and specific configuration options based on your project's requirements.

Sample mobile and web application code

Our solution is a React Native application that combines several libraries and technologies to create an augmented reality (AR) experience with generative AI content, while integrating user authentication using Amplify UI and AWS Cognito. It also incorporates the mobile device's camera for displaying signage within the AR scene. The sample code can be downloaded from https://www.opensourceforu.com/article_codes/aug23/aws.zip.

In the code, do remember to replace with your own values for `YOUR_AWS_REGION`, `YOUR_USER_POOL_ID`, `YOUR_USER_POOL_CLIENT_ID`, `GENERATIVE_AI_API_ENDPOINT`, and ensure that all required libraries and dependencies are properly installed and configured.

Test your React Native app locally to ensure it functions correctly, including authentication and any other desired features.

1. Import required libraries:
 - The 'react' package allows you to define React components.
 - The 'useEffect' and 'useState' hooks are used for managing side effects and state in functional components.
 - The 'View' and 'Text' components are UI components provided by React Native for creating the layout

and displaying text.

- 'PermissionsAndroid' is a React Native API for handling Android permissions.
 - 'Amplify' and 'Auth' are part of the AWS Amplify library, which is used for configuring authentication and authorisation.
 - 'withAuthenticator' is a HOC (higher-order component) provided by `aws-amplify-react-native` to wrap the main component with authentication functionality.
 - 'ViroARSceneNavigator', 'ViroARScene', and 'ViroText' are components from the ViroReact library that enable AR capabilities and rendering AR scenes.
 - 'RNCamera' is a component from the `react-native-camera` package, used to access the mobile device's camera.
2. Configure Amplify:
 - Amplify is configured with the AWS Cognito user pool details, including region, user pool ID, and client ID.
 3. Define the AR Scene component:
 - The 'ARScene' functional component represents the augmented reality scene where the generative AI content will be displayed.
 - The 'generatedContent' state variable is used to store the content fetched from the generative AI API.
 - The 'useEffect' hook is used to make an HTTP request to the generative AI API when the component mounts.
 - The generative AI API is called using the 'fetch' function, and the access token from AWS Cognito is included in the request headers for authorisation.
 - The received content is stored in the 'generatedContent' state variable.
 - The AR scene is rendered using the 'ViroARSceneNavigator' component, which wraps the AR scene content.
 - Within the 'ViroARScene', a 'ViroText' component is used to display the generative AI content as signage in the AR scene.

Continued to page...57